

Initial Project Planning Template

Date	7 July 2026
Team ID	XXXXXX
Project Name	Smart lender
Maximum Marks	5 Marks

1. Sprint Backlog with Acceptance Criteria

Sprint	ID	User Story	Acceptance Criteria	Pts	Priority
Sprint-1	US-01	Clean and impute missing values in the loan dataset.	Missing categoricals filled with column mode, missing numerics filled with column median; zero nulls remain in the processed dataframe.	3	High
Sprint-1	US-02	Encode categorical features into numeric codes.	Gender, Married, Dependents, Education, Self_Employed, Property_Area, Credit_History all map to defined integers per the mappings dict; unmapped values fall back to the encoded mode.	3	High
Sprint-1	US-03	Train and compare Decision Tree, Random Forest, KNN, and XGBoost.	All 4 models are fit on the same 80/20 stratified split; train and test accuracy for each is recorded in an accuracies dict.	5	High
Sprint-1	US-04	Serialize the winning model as a single deployable artifact.	best_model.pkl contains model, scaler, mappings, imputation_values, num_cols, cat_cols, feature_order, accuracies, and loads without error via load_model().	3	High
Sprint-2	US-05	Build the applicant intake form.	/eligibility renders all 11 required fields; submitting POSTs to /predict with correctly named form fields.	3	High
Sprint-2	US-06	Implement the /predict inference route.	Valid form data returns {status:'success', redirect:'/analyzing'}; a missing model returns HTTP 500 with a clear error message.	5	High
Sprint-2	US-07	Return a decision-consistent eligibility score.	Approved results always show score ≥ 50 ; rejected results always show score < 50 , regardless of raw model probability.	3	High
Sprint-2	US-08	Classify each profile into a risk band.	score $\geq 75 \rightarrow$ Low Risk, 60–74 $\rightarrow$ Moderate Risk, $< 60 \rightarrow$ High Risk, each with its matching description text on /results.	2	Medium
Sprint-3	US-09	Generate explainable AI reasons for each decision.	Exactly 4 reason cards render (credit history, DTI coverage, property area, employment/education), each flagged positive or negative.	5	High
Sprint-3	US-10	Compute EMI and repayment breakdown.	Monthly EMI, total interest, and total payable are calculated via the standard amortization formula at 8% annual rate and shown on /results.	3	Medium
Sprint-3	US-11	Generate personalized improvement suggestions.	Approved profiles show ≥ 2 positive suggestions; rejected profiles show suggestions targeted to their weakest factor(s).	3	Medium
Sprint-3	US-12	Add an animated transition between form submit and results.	After submission the user is routed to /analyzing, sees the animation, and is carried to /results without manual action.	2	Low

Sprint-4	US-13	Build the analytics dashboard.	/dashboard shows live record/approved/rejected counts from the CSV (with hardcoded fallback on read failure) and the model accuracy comparison.	5	High
Sprint-4	US-14	Build the EDA page.	/eda renders the pre-generated univariate, bivariate, and multivariate chart images without broken image links.	5	Medium
Sprint-4	US-15	Apply a consistent design system across all pages.	All templates use the shared Tailwind tokens (colors, spacing, type scale) from tailwind.config.js with no conflicting hardcoded styles.	3	Medium
Sprint-4	US-16	Validate the end-to-end user journey.	form → predict → analyzing → results → dashboard → eda completes with zero unhandled exceptions across ≥ 10 varied test profiles.	3	High

2. Technical Task Breakdown

Sprint	ID	Technical Task	File / Module	Est. Hours
Sprint-1	US-01	Strip whitespace from column names/values; compute & store per-column mode/median in imputation_values; apply fillna by column type.	train_pipeline.py	2h
Sprint-1	US-02	Define the mappings dict for all categorical columns; apply .map() with fallback-to-mode handling for unseen values.	train_pipeline.py	2h
Sprint-1	US-03	80/20 stratified split; fit StandardScaler on numeric cols; train all 4 classifiers with their specified hyperparameters; score train/test accuracy.	train_pipeline.py	4h
Sprint-1	US-04	Assemble the model_data dict; pickle.dump to best_model.pkl; implement the lazy-singleton load_model() guard.	train_pipeline.py, app.py	1.5h
Sprint-2	US-05	Build the Tailwind-styled form with fields matching the trained feature_order; wire the form action/method to POST /predict.	loanEligibility.html	4h
Sprint-2	US-06	Read form fields with safe defaults; encode via stored mappings; assemble a single-row DataFrame in feature_order; scale numeric columns.	app.py (/predict)	4h
Sprint-2	US-07	Compute score = int(prob[1]*100); apply the approved/rejected clamping rule so score always agrees with pred.	app.py (/predict)	1h
Sprint-2	US-08	Implement the if/elif risk thresholds; store risk_level/risk_desc in the Flask session.	app.py (/predict)	1h
Sprint-3	US-09	Compute dti_ratio = emi / monthly_income; build the explainable_reasons list; render the 4 cards in the results template.	app.py, prediction.html	3h
Sprint-3	US-10	Implement the amortization EMI formula with a ZeroDivisionError guard; convert loan amount from thousands to currency units.	app.py (/predict)	2h
Sprint-3	US-11	Build the suggestions list, branching on loan_status, credit_history, and a computed suggested_amount.	app.py (/predict)	2h
Sprint-3	US-12	Build the CSS/JS-driven animation with a timed redirect from /analyzing to /results.	AnalzePredictAnimation.html	2h
Sprint-4	US-13	Read & clean the CSV inside /dashboard with a try/except fallback; pass live counts and the accuracies dict to the template.	app.py (/dashboard)	3h
Sprint-4	US-14	Confirm all univariate/bivariate/multivariate PNGs exist under static/images/charts/; wire eda.html sections to those paths.	train_pipeline.py, eda.html	3h
Sprint-4	US-15	Define custom color/spacing/type tokens in tailwind.config.js; apply consistently across index/loanEligibility/dashboard/prediction/eda.	tailwind.config.js, templates/	3h
Sprint-4	US-16	Manually walk all 6 routes; confirm session persists between /predict and /results; confirm graceful handling when best_model.pkl is absent.	app.py, all templates	2h

3. Sprint Delivery Schedule

Sprint	Points	Duration	Start	End (Planned)	End (Actual)	Definition of Done
Sprint-1	9.5h	6 Days	06 Jul 2026	11 Jul 2026	11 Jul 2026	best_model.pkl exists and loads with all 8 expected keys present.
Sprint-2	10h	6 Days	12 Jul 2026	17 Jul 2026	17 Jul 2026	10 varied test profiles submitted through the form each return a valid, decision-consistent score.
Sprint-3	9h	6 Days	18 Jul 2026	23 Jul 2026	23 Jul 2026	/results correctly renders EMI, all 4 explainable reasons, and matching suggestions for both approved and rejected cases.
Sprint-4	11h	6 Days	24 Jul 2026	29 Jul 2026	29 Jul 2026	/dashboard and /eda load without errors and reflect the same dataset/model used by /predict.

4. Dependencies & Risks

Dependency / Risk	Impact	Mitigation
US-04 depends on US-01, US-02, US-03	Model, scaler, and mappings can't be serialized before preprocessing and training are finished.	Sequence Sprint-1 stories in order; do not start US-04 until US-03's accuracies dict is populated.
US-06 depends on US-04	The /predict route needs best_model.pkl to exist and load before it can run inference.	Run train_pipeline.py once at the start of Sprint-2 and re-run it any time the dataset or mappings change.
US-09, US-10, US-11 depend on US-06, US-07, US-08	Explainable reasons, EMI, and suggestions all read values (pred, prob, emi, credit_history, etc.) produced during /predict.	Keep all derived values in the same session-write block in /predict so downstream stories have a single source of truth.
US-13 depends on US-04	The dashboard's model comparison chart reads the accuracies dict saved inside best_model.pkl.	If best_model.pkl is regenerated, verify /dashboard still renders before considering Sprint-4 done.
Risk: missing/corrupted best_model.pkl	load_model() logs a warning but the app keeps running; /predict will then fail at runtime.	Re-run train_pipeline.py and confirm the file is present at the same directory level as app.py before demoing.
Risk: dataset column drift	Renamed or reordered CSV columns would break the fixed feature_order used for inference.	Keep the existing .columns.str.strip() normalization and re-validate feature_order after any dataset change.

5. Sprint-to-Feature Traceability

Sprint	What Ships
Sprint-1	train_pipeline.py end-to-end: preprocessing, encoding, 4-model comparison, best_model.pkl.
Sprint-2	app.py /predict + loanEligibility.html: decision-consistent score and risk band.
Sprint-3	prediction.html + /predict extensions: explainable reasons, EMI calculator, suggestions, analyzing animation.
Sprint-4	dashboard.html, eda.html, tailwind.config.js: analytics, EDA charts, unified design system, final validation.